



PostgreSQL Database Fundamentals

Eng. Fatma Elraey

Day 3 — Altering Tables, Sorting, Grouping, and Aliases

ALTER TABLE • ORDER BY • GROUP BY / HAVING • LIMIT / OFFSET • Aliases

Recap: Where We Left Off

Day 1 covered creating databases and tables and basic CRUD. Day 2 covered constraints (CHECK, UNIQUE, DEFAULT, PRIMARY KEY, FOREIGN KEY) and referential integrity. Day 3 focuses on reshaping a table after it already exists, and on asking richer questions of the data already inside it.

Section 2: ALTER TABLE — Modifying Table Structure

Why ALTER TABLE?

Tables rarely stay exactly as first designed. ALTER TABLE changes an existing table's structure — adding, changing, or removing columns and constraints — without losing the data already stored in it.

Adding a Column

SQL

```
\c university
You are now connected to database "university" as user "postgres".
```

SQL

```
\dt
-- public | courses | table | postgres
-- public | doctors | table | postgres
-- public | employees | table | postgres
-- public | items | table | postgres
-- public | orders | table | postgres
-- public | products | table | postgres
-- public | students | table | postgres
```

SQL

```
SELECT * FROM students;
-- std_id | std_name | address
-- -----+-----+-----
--      1 | mohamed | egypt
--      2 | sara    | qatar
```

Add a new column called gender:

SQL

```
ALTER TABLE students ADD gender integer;
-- ALTER TABLE
```

SQL

```
SELECT * FROM students;
-- std_id | std_name | address | gender
-- -----+-----+-----+-----
--      1 | mohamed | egypt   |
--      2 | sara    | qatar   |
```

 A newly added column starts out NULL for every existing row, since PostgreSQL has no value to fill in automatically unless a DEFAULT is specified.

Changing a Column's Data Type

PostgreSQL can convert a column to a new type as long as the existing values can be reinterpreted in that type.

SQL

```
ALTER TABLE students ALTER COLUMN age TYPE text;
-- ALTER TABLE
```

 Numbers convert to text easily, since every number has a valid text representation. The reverse is not automatic — converting text to a numeric type fails if any stored value is not a valid number, and often needs an explicit USING expression

SQL

Dropping a Column

SQL

```
ALTER TABLE students DROP COLUMN gender;
-- ALTER TABLE
```

SQL

```
SELECT * FROM students;
-- std_id | std_name | address | age
-- -----+-----+-----+-----
--      1 | mohamed  | egypt   |
--      2 | ali      | qatar   |
--      3 | sara     | egypt   |
--      4 | sara     | dupai   | 20
--      5 | jkjk     | dupai   | 50
```

 **Warning:** *DROP COLUMN permanently removes that column and every value stored in it. There is no undo without a backup.*

Adding a Constraint to an Existing Table

A primary key can be added after the table already exists, by naming the constraint:

SQL

```
ALTER TABLE students ADD CONSTRAINT pk PRIMARY KEY (std_id);
-- ALTER TABLE
```

Naming the constraint (pk in this example) makes it easy to refer to later, for example when dropping it.

Dropping a Constraint

Incorrect: repeating the constraint definition when dropping it is not valid syntax.

SQL

```
ALTER TABLE students DROP CONSTRAINT pk PRIMARY KEY (std_id);
-- ERROR: syntax error
```

Correct: only the constraint name is needed.

SQL

```
ALTER TABLE students DROP CONSTRAINT pk;
-- ALTER TABLE
```

Renaming a Table

SQL

```
ALTER TABLE students RENAME TO members;
-- ALTER TABLE
```

After renaming, the old table name no longer exists:

SQL

```
SELECT * FROM students;
```

```
ERROR:  relation "students" does not exist
LINE 1: select * from students;
```

! Important: every later query in this lesson uses members, since students was renamed. Always check \dt if a table name seems to be missing.

Other Useful ALTER TABLE Operations

Beyond the operations already shown, these are commonly used and worth knowing:

Goal	Statement
Rename a column	ALTER TABLE members RENAME COLUMN std_name TO full_name;
Add a default to an existing column	ALTER TABLE members ALTER COLUMN address SET DEFAULT 'egypt';
Remove a column's default	ALTER TABLE members ALTER COLUMN address DROP DEFAULT;
Require a value going forward	ALTER TABLE members ALTER COLUMN age SET NOT NULL;
Allow NULL again	ALTER TABLE members ALTER COLUMN age DROP NOT NULL;

Section 3: Updating and Deleting Data

Updating Rows

```
SQL
```

```
UPDATE members SET age = 20 WHERE std_id = 1;
-- UPDATE 1
```

```
SQL
```

```
SELECT * FROM members;
-- std_id | std_name | address | age
-- -----+-----+-----+-----
--      2 | ali      | qatar   |
--      3 | sara     | egypt   |
--      4 | sara     | dupai   | 20
--      5 | jkjk     | dupai   | 50
--      1 | mohamed  | egypt   | 20
```

```
SQL
```

```
UPDATE members SET address = 'usa' WHERE std_id = 2;
-- UPDATE 1
```

⚠ Warning: an UPDATE without a WHERE clause changes every row in the table. This example intentionally sets every row's age to 20:

```
SQL
```

```
UPDATE members SET age = 20;
-- UPDATE 5
```

Deleting Rows

```
SQL

DELETE FROM members WHERE std_name = 'sara';
-- or, to also catch names containing sara:
DELETE FROM members WHERE std_name LIKE '%sara%';
```

```
SQL

DELETE FROM members WHERE std_id = 4;
-- DELETE 1
```

💡 Recommendation: run the equivalent *SELECT* first to confirm exactly which rows will be changed or removed, especially with *LIKE* patterns.

Section 4: Sorting Data with ORDER BY

ASC and DESC

ORDER BY controls the sequence of returned rows. ASC (ascending) is the default; DESC (descending) reverses it.

```
SQL

SELECT * FROM employee ORDER BY id ASC;
-- 1 | Alice | 25 | 50000
-- 2 | Bob   | 30 | 60000
-- 3 | Charlie | 28 | 55000
-- ... (continues in ascending id order)

SQL

SELECT * FROM employee ORDER BY id DESC;
-- 13 | ali | 67 | 67000
-- 12 | ahmed | | 67000
-- 11 | fatma | 24 |
-- ... (continues in descending id order)

SQL

SELECT * FROM employee ORDER BY name ASC;
-- 12 | ahmed | | 67000
-- 13 | ali | 67 | 67000
-- 1 | Alice | 25 | 50000
-- ... (alphabetical by name)
```

🔗 PostgreSQL's default sort treats lowercase and uppercase letters according to the database locale, which is why *ahmed* and *ali* sort before *Alice* in this result.

Sorting by Multiple Columns

Extra columns act as tie-breakers when the first column has repeated values:

```
SQL
SELECT * FROM members ORDER BY address ASC, age DESC;
```

Rows are sorted by address first; whenever two rows share the same address, age (descending) decides their order.

Sorting and NULL Values

By default, PostgreSQL sorts NULL values last in ascending order and first in descending order. This can be made explicit:

```
SQL
SELECT * FROM members ORDER BY age ASC NULLS LAST;
SELECT * FROM members ORDER BY age ASC NULLS FIRST;
```

Section 5: Aggregate Functions

What Are Aggregate Functions?

An aggregate function takes many rows and reduces them to a single summary value. They are the foundation GROUP BY builds on.

Function	Returns
COUNT(*)	The number of rows
SUM(column)	The total of a numeric column
AVG(column)	The average of a numeric column
MIN(column)	The smallest value
MAX(column)	The largest value

Aggregate Functions Without GROUP BY

Used alone, an aggregate function summarizes the entire result set into one row:

```
SQL
SELECT COUNT(*) FROM members;
SELECT AVG(salary) FROM members;
SELECT MAX(age), MIN(age) FROM members;
```

Section 6: Grouping Data with GROUP BY and HAVING

What Is GROUP BY?

GROUP BY collapses rows that share the same value in one or more columns into a single summary row per group, typically paired with an aggregate function.

Grouping by One Column

```
SQL
SELECT address, COUNT(*) FROM members GROUP BY address;
-- address | count
-- -----+-----
-- dupai   |      4
-- egypt   |      2
-- usa     |      1
```

Grouping by Multiple Columns

```
SQL
SELECT address, age, COUNT(*) FROM members GROUP BY address, age;
-- address | age | count
-- -----+-----+-----
-- dupai   | 50 |      2
-- usa     | 20 |      1
-- dupai   | 40 |      1
-- dupai   | 20 |      1
-- egypt   | 34 |      1
-- egypt   | 20 |      1
```

Each distinct combination of address and age becomes its own group.

Filtering Groups with HAVING

WHERE filters individual rows before grouping happens; HAVING filters entire groups after the aggregate has been calculated.

```
SQL
SELECT address, age, COUNT(*)
FROM members
GROUP BY address, age
HAVING SUM(salary) > 3000;
```

⚠ Important: WHERE cannot reference an aggregate function such as SUM(salary) directly, because WHERE runs before grouping. Use HAVING for conditions on aggregated values.

WHERE versus HAVING

Clause	Filters	Runs
WHERE	Individual rows	Before grouping
HAVING	Groups (usually via an aggregate)	After grouping

Section 7: LIMIT and OFFSET

LIMIT: Restricting the Number of Rows

LIMIT returns only the first matching rows, which is useful for previews and pagination.

SQL

```
SELECT * FROM members ORDER BY std_id;
-- returns every row, ordered by std_id
```

SQL

```
SELECT * FROM members ORDER BY std_id LIMIT 6;
-- returns only the first 6 rows in that order
```

! Important: without ORDER BY, PostgreSQL does not guarantee any particular row order, so LIMIT alone can return a different subset each time the query runs. Always pair LIMIT with ORDER BY when the exact rows matter.

OFFSET: Skipping Rows

OFFSET skips a number of rows before starting to return results — the basis of pagination.

SQL

```
SELECT * FROM members ORDER BY std_id LIMIT 6 OFFSET 4;
-- skips the first 4 rows, then returns up to 6 more
```

SQL

```
SELECT * FROM members ORDER BY std_id LIMIT 6 OFFSET 8;
-- skips the first 8 rows, then returns up to 6 more
```

Building a Simple Pagination Pattern

A common pattern for showing results page by page (page size of 6 rows):

Page	Query
1	LIMIT 6 OFFSET 0
2	LIMIT 6 OFFSET 6
3	LIMIT 6 OFFSET 12
 General formula: $OFFSET = (page_number - 1) * page_size$.	

Section 8: Aliases and Concatenation

What Is an Alias?

An alias renames a column or table for the duration of a query's output. It exists only in the result — it is never stored in the database.

Column Aliases

SQL

```
SELECT std_name FROM members;
-- column heading: std_name
```

SQL

```
SELECT std_name AS first_name FROM members;
-- column heading: first_name
```

Multi-Word Aliases Need Double Quotes

An alias with a space is not valid on its own:

SQL

```
SELECT std_name AS first name FROM members;
```

```
ERROR:  syntax error at or near "name"
```

Wrapping it in double quotes fixes it:

SQL

```
SELECT std_name AS "first name" FROM members;
-- column heading: first name
```

✎ Rule of thumb: single quotes ('...') hold text data, double quotes ("...") wrap identifiers such as column names, table names, and aliases that need special characters or spaces.

Concatenating Columns

The || operator joins text values together:

SQL

```
SELECT std_name || age FROM members;
-- ?column?
-- -----
-- jkjk20
-- mohamed20
```

Naming the result with an alias makes it readable:

SQL

```
SELECT std_name || age AS "student age" FROM members;
```

Adding a literal space between the parts makes the output more natural:

SQL

```
SELECT std_name || ' ' || age AS "student age" FROM members;
-- student age
```

```
-- -----
-- jkjk 20
-- mohamed 20
```

✎ Using double quotes instead of single quotes for the space breaks the query, because double quotes are read as an identifier, not as text:

```
SQL
```

```
SELECT std_name || " " || age AS "student age" FROM members;
```

```
ERROR: column " " does not exist
```

Table Aliases

A table can also be given a short alias, useful in longer queries and essential once multiple tables are joined:

```
SQL
```

```
SELECT std_id, std_name, age
FROM members m
WHERE m.std_id > 6;
```

m now stands in for members, so m.std_id means "the std_id column of the members table."

Section 9: DISTINCT

Removing Duplicate Rows

DISTINCT removes duplicate rows from a result, keeping only one copy of each unique combination of selected columns.

```
SQL
```

```
SELECT address FROM members;
-- may repeat the same address many times
```

```
SQL
```

```
SELECT DISTINCT address FROM members;
-- each address appears only once
```

DISTINCT can also apply to a combination of columns:

```
SQL
```

```
SELECT DISTINCT address, age FROM members;
-- each unique address/age pair appears only once
```

Putting It All Together

A single query can combine filtering, grouping, aggregation, sorting, and pagination:

```
SQL
```

```
SELECT address, COUNT(*) AS members_count
FROM members
WHERE age IS NOT NULL
GROUP BY address
HAVING COUNT(*) > 1
ORDER BY members_count DESC
LIMIT 5;
```

🔗 *Reading order of execution: FROM and WHERE run first, then GROUP BY, then HAVING, then the values are sorted with ORDER BY, and finally LIMIT trims the result.*

Command Summary

```
SQL

ALTER TABLE members ADD gender integer;
ALTER TABLE members ALTER COLUMN age TYPE text;
ALTER TABLE members DROP COLUMN gender;
ALTER TABLE members ADD CONSTRAINT pk PRIMARY KEY (std_id);
ALTER TABLE members DROP CONSTRAINT pk;
ALTER TABLE members RENAME TO members;

SELECT * FROM members ORDER BY age DESC;
SELECT address, COUNT(*) FROM members GROUP BY address;
SELECT address, COUNT(*) FROM members GROUP BY address HAVING COUNT(*) > 1;
SELECT * FROM members ORDER BY std_id LIMIT 6 OFFSET 4;
SELECT std_name AS "first name" FROM members;
SELECT DISTINCT address FROM members;
```

Knowledge Check

1. What happens to existing rows when a new column is added with ALTER TABLE ADD?
2. Why does converting a numeric column to text always work, while the reverse might not?
3. What is wrong with ALTER TABLE members DROP CONSTRAINT pk PRIMARY KEY (std_id)?
4. Why should ORDER BY usually be paired with LIMIT?
5. What is the difference between WHERE and HAVING?
6. How would you show page 3 of results, 10 rows per page, using LIMIT and OFFSET?
7. Why does an alias need double quotes when it contains a space?
8. What does DISTINCT do to duplicate rows in a result?

Lab Safety Exercise

? Before running an UPDATE or DELETE without a WHERE clause on a shared table, always confirm with your instructor or team first.

⚠️ *Expected answer: an UPDATE or DELETE with no WHERE clause affects every row in the table at once, which is rarely intentional and cannot be undone without a backup.*

Practical Challenge

Write a single query that reports, for each city, the number of members and their average salary, but only for cities with at least 2 members, sorted by member count in descending order, showing only the top 3 cities.

SQL

```
SELECT city, COUNT(*) AS member_count, AVG(salary) AS avg_salary
FROM members
GROUP BY city
HAVING COUNT(*) >= 2
ORDER BY member_count DESC
LIMIT 3;
```

codes

Part 1: ALTER TABLE

SQL

```
ALTER TABLE students ADD gender integer;
ALTER TABLE students ALTER COLUMN age TYPE text;
ALTER TABLE students DROP COLUMN gender;
ALTER TABLE students ADD CONSTRAINT pk PRIMARY KEY (std_id);
ALTER TABLE students DROP CONSTRAINT pk;
ALTER TABLE students RENAME TO members;
```

Part 2: Update and Delete

SQL

```
UPDATE members SET age = 20 WHERE std_id = 1;
UPDATE members SET address = 'usa' WHERE std_id = 2;
DELETE FROM members WHERE std_name = 'sara';
DELETE FROM members WHERE std_id = 4;
```

Part 3: ORDER BY

SQL

```
SELECT * FROM employee ORDER BY id ASC;
SELECT * FROM employee ORDER BY id DESC;
SELECT * FROM employee ORDER BY name ASC;
SELECT * FROM members ORDER BY address ASC, age DESC;
```

Part 4: Aggregate Functions and GROUP BY

SQL

```
SELECT COUNT(*) FROM members;
SELECT AVG(salary) FROM members;
SELECT address, COUNT(*) FROM members GROUP BY address;
SELECT address, age, COUNT(*) FROM members GROUP BY address, age;
SELECT address, age, COUNT(*)
```

```
FROM members
GROUP BY address, age
HAVING SUM(salary) > 3000;
```

Part 5: LIMIT and OFFSET

```
SQL

SELECT * FROM members ORDER BY std_id;
SELECT * FROM members ORDER BY std_id LIMIT 6;
SELECT * FROM members ORDER BY std_id LIMIT 6 OFFSET 4;
SELECT * FROM members ORDER BY std_id LIMIT 6 OFFSET 8;
```

Part 6: Aliases, Concatenation, DISTINCT

```
SQL

SELECT std_name AS first_name FROM members;
SELECT std_name AS "first name" FROM members;
SELECT std_name || ' ' || age AS "student age" FROM members;
SELECT std_id, std_name, age FROM members m WHERE m.std_id > 6;
SELECT DISTINCT address FROM members;
```

The Semicolon Is Required

Every SQL statement in PostgreSQL must end with a semicolon. Without it, psql assumes the statement is not finished yet and waits for more input on the next line — it does not execute anything.

```
SQL

test=# SELECT * FROM employees
test-# ;
-- psql keeps prompting with test-# until the semicolon arrives
```

✎ Unlike some other database tools where the semicolon is a style choice, in PostgreSQL it is not optional — a missing semicolon simply means the statement has not been sent yet.

Column Aliases: AS Is Optional

The AS keyword makes an alias easier to read, but PostgreSQL does not require it. Both statements below produce identical output:

```
SQL

SELECT model AS car FROM cars;
SELECT model car FROM cars;
-- both return a column headed "car"
```

💡 Recommendation: keep AS in scripts anyway — it is clearer to read and makes the alias obvious at a glance, especially in longer queries.

Concatenation Uses ||, Not +

PostgreSQL concatenates text with the `||` operator. The `+` operator is reserved for arithmetic and does not join text values.

```

SQL
SELECT product_name || unit AS product FROM products;
-- correct: joins the two text values

SQL
SELECT product_name + unit AS product FROM products;
-- incorrect: + is not a text operator in PostgreSQL

```

A Quick Operator Recap

PostgreSQL supports a specific, standard set of operators. Keywords borrowed from other tools — such as `TOP`, `SORT`, or `RANGE` — do not exist in PostgreSQL.

Category	Valid PostgreSQL Operators
Comparison	<code>=</code> , <code><</code> , <code>></code> , <code><=</code> , <code>>=</code> , <code><></code> , <code>!=</code>
Logical	<code>AND</code> , <code>OR</code> , <code>NOT</code>
Pattern matching	<code>LIKE</code> (case-sensitive), <code>ILIKE</code> (case-insensitive)
NULL testing	<code>IS NULL</code> , <code>IS NOT NULL</code>
Range	<code>BETWEEN ... AND ...</code>
Set membership	<code>IN (...)</code>

⚠ `IS`, `LIKE`, `AND`, and `ILIKE` are all valid in PostgreSQL — there is no trick keyword among them to avoid.

Section 2: Auto-Incrementing Columns — SERIAL vs IDENTITY

Recap: GENERATED ALWAYS AS IDENTITY

Day 1 introduced the modern, SQL-standard way to auto-generate a primary key:

```

SQL
employee_id integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY

```

The Classic Alternative: SERIAL

Older PostgreSQL code — and many tutorials — use the `SERIAL` keyword instead. `SERIAL` is shorthand that creates an integer column backed by an auto-incrementing sequence.

```

SQL

CREATE TABLE cars (
    car_id SERIAL PRIMARY KEY,
    model  varchar(50)
);

```

Keyword	Storage	Typical Use
smallserial	2 bytes	Small tables, few rows expected
serial	4 bytes	Most tables
bigserial	8 bytes	Very large tables

SERIAL versus IDENTITY

Aspect	SERIAL	GENERATED ALWAYS AS IDENTITY
SQL standard?	No — a PostgreSQL-specific shorthand	Yes
Underlying mechanism	An implicit sequence with a DEFAULT	An implicit sequence, managed as an identity
Recommended for new code?	Still common, especially in older code	Yes — preferred in modern PostgreSQL

💡 *Recommendation: recognize SERIAL when reading existing code or exam questions, but prefer GENERATED ALWAYS AS IDENTITY when writing new tables.*

Section 3: Removing Data — DELETE vs TRUNCATE vs DROP

Three Ways to Remove Rows or Tables

Statement	Removes	Structure Kept?	Can Filter with WHERE?
DELETE FROM cars WHERE ...;	Selected rows, or all rows	Yes	Yes
TRUNCATE TABLE cars;	All rows, quickly	Yes	No
DROP TABLE cars;	The entire table and its data	No	No

Statement	Removes	Structure Kept?	Can Filter with WHERE?
✂ There is no <code>DELETE TABLE</code> statement in PostgreSQL — <code>DELETE</code> always removes rows (<code>DELETE FROM table</code>), while removing an entire table's contents in one step uses <code>TRUNCATE</code>, and removing the table itself uses <code>DROP</code>. <pre> -- SQL TRUNCATE TABLE cars; -- removes every row from cars, -- keeps the table structure </pre>			

Recap: The DELETE Return Message

Deleting one row from a table always reports how many rows were affected:

```

-- SQL

DELETE FROM cars WHERE car_id = 3;
-- DELETE 1

```

Section 4: Why We Need JOINS

The Problem: Related Data Lives in Separate Tables

Day 2 showed that a foreign key connects an orders row to a products row without duplicating the product's details inside orders. That keeps data consistent, but it also means a single orders row cannot show a product's name by itself.

```

-- SQL

SELECT * FROM orders;
-- order_id | product_id | quantity
-- -----+-----+-----
--      1 |          1 |         6

```

To see the product's name alongside the order, the two tables must be combined — that is what a JOIN does.

Sample Tables Used in This Section

```

-- SQL

CREATE TABLE products (
  product_id integer PRIMARY KEY,
  name       text,
  price      numeric
);

-- SQL

CREATE TABLE orders (

```

```

    order_id    integer PRIMARY KEY,
    product_id  integer REFERENCES products(product_id),
    quantity    numeric
);

```

SQL

```

INSERT INTO products VALUES (1, 'tea', 10), (2, 'coffee', 9), (3, 'sugar', 5);
INSERT INTO orders VALUES (1, 1, 6), (2, 2, 7), (3, 1, 2);
-- note: no order references product_id 3 (sugar) yet

```

Section 5: INNER JOIN

What INNER JOIN Does

INNER JOIN returns only the rows that have a match in both tables. A product with no matching order, or an order with no matching product, is left out.

SQL

```

SELECT o.order_id, p.name, o.quantity
FROM orders o
INNER JOIN products p ON o.product_id = p.product_id;
-- order_id | name    | quantity
-- -----+-----+-----
--          1 | tea     |        6
--          2 | coffee  |        7
--          3 | tea     |        2

```

Sugar (product_id 3) does not appear, because no order references it.

Section 6: LEFT, RIGHT, and FULL JOIN

LEFT JOIN: Keep Everything from the Left Table

LEFT JOIN returns every row from the left table, matched with the right table where possible — filling in NULL when there is no match.

SQL

```

SELECT p.name, o.order_id, o.quantity
FROM products p
LEFT JOIN orders o ON p.product_id = o.product_id;
-- name    | order_id | quantity
-- -----+-----+-----
-- tea     |          |        6
-- coffee  |          |        7
-- tea     |          |        2
-- sugar   |          |

```

Sugar now appears, with NULL in the order columns, because LEFT JOIN keeps every product regardless of whether it was ordered.

RIGHT JOIN: Keep Everything from the Right Table

RIGHT JOIN is the mirror image of LEFT JOIN — every row from the right table is kept, matched with the left table where possible.

SQL

```
SELECT p.name, o.order_id, o.quantity
FROM orders o
RIGHT JOIN products p ON o.product_id = p.product_id;
-- produces the same result as the LEFT JOIN example above,
-- because products is now on the right
```

 In practice, most PostgreSQL developers write LEFT JOIN and simply reorder the tables, rather than using RIGHT JOIN — the two are interchangeable if you swap which table is listed first.

FULL JOIN: Keep Everything from Both Tables

FULL JOIN (or FULL OUTER JOIN) returns every row from both tables, matching where possible and filling NULL on whichever side has no match.

SQL

```
SELECT p.name, o.order_id
FROM products p
FULL JOIN orders o ON p.product_id = o.product_id;
-- includes matched rows, unmatched products (NULL order), and
-- unmatched orders (NULL product), if any existed
```

Comparing the Join Types

Join Type	Keeps
INNER JOIN	Only rows matched in both tables
LEFT JOIN	All left-table rows, matched right-table rows or NULL
RIGHT JOIN	All right-table rows, matched left-table rows or NULL
FULL JOIN	All rows from both tables, matched or NULL on either side

Section 7: CROSS JOIN

What CROSS JOIN Does

CROSS JOIN pairs every row in the first table with every row in the second table — there is no matching condition. This is called a Cartesian product.

SQL

```
SELECT p.name, o.order_id
FROM products p
```

```
CROSS JOIN orders o;
```

Predicting the Row Count

A CROSS JOIN always returns the number of rows in the first table multiplied by the number of rows in the second table.

```
SQL
-- products has 10 rows, orders
has 8 rows
SELECT * FROM products CROSS JOIN
orders;
-- returns 10 x 8 = 80 rows
```

Left Table Rows	Right Table Rows	CROSS JOIN Result Rows
10	8	80
3	3	9
5	1	5

Warning: CROSS JOIN grows very quickly with table size and is rarely what you want for everyday reporting — it is normally used deliberately, for example to generate every possible combination of two small lists.

Command Summary

```
SQL
-- Alias without AS
SELECT model car FROM cars;

-- SERIAL column
car_id SERIAL PRIMARY KEY

-- Removing data
DELETE FROM cars WHERE car_id = 3;
TRUNCATE TABLE cars;
DROP TABLE cars;

-- Joins
SELECT o.order_id, p.name FROM orders o INNER JOIN products p ON o.product_id =
p.product_id;
SELECT p.name, o.order_id FROM products p LEFT JOIN orders o ON p.product_id = o.product_id;
SELECT p.name, o.order_id FROM products p FULL JOIN orders o ON p.product_id = o.product_id;
SELECT p.name, o.order_id FROM products p CROSS JOIN orders o;
```

Knowledge Check

9. What happens in psql if you press Enter after a SELECT statement without a semicolon?

10. Do `SELECT model AS car FROM cars` and `SELECT model car FROM cars` return different results?
11. Why does `SELECT product_name + unit AS product FROM products;` fail in PostgreSQL?
12. What is the difference between `SERIAL` and `GENERATED ALWAYS AS IDENTITY`?
13. Why is there no `DELETE TABLE` statement in PostgreSQL?
14. If `products` has 10 rows and `orders` has 8 rows, how many rows does a `CROSS JOIN` between them return?
15. Which join keeps every row from `products` even when a product has no matching order?

Lab Safety Exercise

? Before running `TRUNCATE` or `DROP` on a shared table, confirm with `SELECT COUNT(*)` how many rows currently exist.

⚠ Expected answer: `TRUNCATE` and `DROP` cannot be filtered with `WHERE` and cannot be undone without a backup, so confirming the row count first avoids losing data by mistake.

Practical Challenge

Write a single query using `LEFT JOIN`, `GROUP BY`, and `HAVING` that lists every product together with how many times it has been ordered, including products that have never been ordered, showing only products ordered 2 or more times.

```
SQL
SELECT p.name, COUNT(o.order_id) AS times_ordered
FROM products p
LEFT JOIN orders o ON p.product_id = o.product_id
GROUP BY p.name
HAVING COUNT(o.order_id) >= 2;
```

codes

Part 1: SQL Essentials

```
SQL
SELECT model AS car FROM cars;
SELECT model car FROM cars;
SELECT product_name || unit AS product FROM products;
```

Part 2: SERIAL

```
SQL
CREATE TABLE cars (
    car_id SERIAL PRIMARY KEY,
    model varchar(50)
);
INSERT INTO cars (model) VALUES ('Sedan'), ('Hatchback'), ('SUV');
```

```
SELECT * FROM cars;
```

Part 3: DELETE, TRUNCATE, DROP

```
SQL
```

```
DELETE FROM cars WHERE car_id = 3;
TRUNCATE TABLE cars;
DROP TABLE cars;
```

Part 4: Building the Join Tables

```
SQL
```

```
CREATE TABLE products (
  product_id integer PRIMARY KEY,
  name       text,
  price      numeric
);
CREATE TABLE orders (
  order_id   integer PRIMARY KEY,
  product_id integer REFERENCES products(product_id),
  quantity   numeric
);
INSERT INTO products VALUES (1, 'tea', 10), (2, 'coffee', 9), (3, 'sugar', 5);
INSERT INTO orders VALUES (1, 1, 6), (2, 2, 7), (3, 1, 2);
```

Part 5: Joins

```
SQL
```

```
SELECT o.order_id, p.name, o.quantity
FROM orders o
INNER JOIN products p ON o.product_id = p.product_id;

SELECT p.name, o.order_id, o.quantity
FROM products p
LEFT JOIN orders o ON p.product_id = o.product_id;

SELECT p.name, o.order_id
FROM products p
FULL JOIN orders o ON p.product_id = o.product_id;

SELECT p.name, o.order_id
FROM products p
CROSS JOIN orders o;
```